

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**

PRINCIPLES OF DATABASE MANAGEMENT
IT079IU

FINAL REPORT

Topic: Video Streaming Database

Source code: <https://github.com/iamwujiabao/streamflix>

Team Members:

Võ Gia Bảo – ITCSIU24010 – Team Leader

Nguyễn Quốc Trung Kiên – ITCSIU24046 – Team Member

Trần Đức Mạnh – ITCSIU22303 – Team Member

Nguyễn Minh Nhật – ITITWE24060 – Team Member

Trần Đức Phong – ITITI22123 – Team Member

Dương Hoàng Thiên Phúc – ITITI24046 – Team Member

Nguyễn Hà An Thạnh – ITITWE22051 – Team Member

Võ Ngọc Anh Thư – ITITI23036 – Team Member

Instructor: Assoc. Prof. Nguyen Thi Thuy Loan, PhD

Academic Year 2025 – 2026

ABSTRACT

This report documents the design and implementation of StreamFlix, a relational database system that powers an online video-streaming platform. The project is a comprehensive application of the concepts covered in the IT079IU course – conceptual modelling with Entity-Relationship diagrams, translation to the relational model, normalization up to Boyce-Codd Normal Form, writing SQL queries using relational algebra principles, and building a working application on top of the database.

StreamFlix provides the core capabilities expected from a modern streaming service: users can register and subscribe to paid plans; content creators can manage channels and upload videos organized by categories and tags; viewers can watch videos, like or dislike them, leave comments and replies, build personal playlists, and receive notifications. A recommendation engine surfaces videos based on collaborative filtering. Every one of the seven components requested in the project brief — video content management, video storage, user profiles and authentication, streaming and delivery, search and recommendation engine, user interaction, and content management — is represented by one or more tables, enforced with referential integrity, and exposed through a REST API.

The database is built on MySQL 8, containing 16 tables wholly in third normal form (and mostly in Boyce-Codd Normal Form) with five triggers maintaining denormalized counters and two analytical views. The backend is implemented in Java 21 with Spring Boot 3, employing Spring Data JPA for persistence and Spring Security with stateless JWT authentication. A JavaFX 21 desktop client exercises every REST endpoint. The system is packaged as a multi-stage Docker image and orchestrated with docker-compose for production deployment. The report covers the full lifecycle of the project: the conceptual ER diagram, step-by-step translation to the relational schema, a formal normalization analysis, twenty-five representative SQL queries — from basic SELECT statements to advanced CTEs and window functions — and finally, the software architecture that turns the database into a usable streaming service.

Keywords: relational database, ER modelling, normalization, BCNF, SQL, Spring Boot, video streaming.

TABLE OF CONTENTS

ABSTRACT.....	3
LIST OF TABLES.....	6
LIST OF FIGURES.....	7
CHAPTER 1. INTRODUCTION.....	8
1.1 System Overview.....	8
1.2 Project Goals.....	8
1.3 Techniques and Tools Used.....	8
1.4 Scope and Limitations.....	9
1.5 Structure of the Report.....	9
CHAPTER 2. TASK TIMELINE AND DIVISION.....	11
2.1 Project Timeline.....	11
2.2 Task Allocation.....	11
CHAPTER 3. METHODOLOGY.....	13
3.1 Requirements and Entity Identification.....	13
3.2 Entity-Relationship Diagram.....	13
3.3 Translation to the Relational Model.....	16
3.4 Normalization Analysis.....	18
3.5 Keys and Functional Dependencies.....	20
3.6 Relational Algebra Foundations.....	20
CHAPTER 4. IMPLEMENTATION AND DISCUSSION.....	22
4.1 System Architecture.....	22
4.2 Database Implementation.....	23
4.3 SQL Queries Catalogue.....	24
4.4 Backend Implementation.....	25
4.5 Frontend.....	26
4.6 Testing and Validation.....	27
4.7 Results Obtained.....	27
4.8 Challenges and Solutions.....	27
CHAPTER 5. CONCLUSION AND FUTURE WORK.....	29
5.1 Summary of Key Findings.....	29
5.2 Reflection on Objectives.....	29

5.3 Implications.....	29
5.4 Future Work.....	30
5.5 Closing Remarks.....	30
REFERENCES.....	31
APPENDICES.....	32

LIST OF TABLES

Table 2.1 — Task allocation matrix

Table 2.2 — Project timeline

Table 3.1 — Strong entities and primary keys

Table 3.2 — Relationships and cardinalities

Table 3.3 — Final relational schema

Table 3.4 — Normal form verdict per table

Table 4.1 — Sample query result: top videos per channel

Table 4.2 — REST API endpoints

LIST OF FIGURES

Figure 3.1 — High-level Entity-Relationship Diagram

Figure 3.2 — Relational schema diagram

Figure 3.3 — System architecture layers

Figure 4.1 — Relational algebra tree for Query Q19 (top-3 per channel)

Figure 4.2 — Trigger flow for view-count maintenance

Figure 4.3 — Home screen of the demo web application

CHAPTER 1. INTRODUCTION

1.1 System Overview

StreamFlix is a prototype video-streaming platform built entirely around a relational database. The system models the primary use cases of services such as YouTube and Netflix: authenticated users browse a catalogue of videos, hit the play button, leave reactions and comments, subscribe to the channels of creators they enjoy, and follow curated recommendations. Creators, in turn, upload content to their channels, tag it with categories and free-form keywords, and monitor how their audience responds.

Although the application layer is intentionally light — a Java backend with a JavaFX desktop client — the database layer is production-grade: sixteen carefully modelled tables with full referential integrity, triggers that keep denormalized counters in sync, and indexes (including a full-text index) tuned for the access patterns of a real streaming platform. The emphasis throughout the project is on the database itself, in line with the learning outcomes of IT079IU.

1.2 Project Goals

The project translates every learning outcome of the course into a concrete, working deliverable:

1. CLO1 — Develop an (Extended) ER model from requirements. Chapter 3 presents the ER diagram for the full streaming domain, with the associated entity types, weak entities, relationship cardinalities, and an ISA hierarchy over the user role.
2. CLO2 — Apply normalization principles to transform an ER model into a relational schema. The ER diagram is translated step-by-step using the seven-step algorithm and every relation is then analysed against 1NF through BCNF, with intentional denormalization clearly justified.
3. CLO3 — Construct efficient SQL queries, relational algebra expressions, and query trees for data retrieval and manipulation. Twenty-five queries of increasing sophistication — from simple SELECT statements to multi-level CTEs with window functions — are shipped, and representative queries are illustrated with their relational algebra trees.

1.3 Techniques and Tools Used

The system is built with tools commonly used in industry:

- MySQL 8 as the primary relational database management system, with triggers, views, generated columns, and a full-text index.
- Java 21 and Spring Boot 3 for the backend, applying the layered architecture (controller → service → repository → entity) popular in enterprise Java.
- Spring Data JPA / Hibernate for object-relational mapping with support for composite keys, self-relationships, and many-to-many associations.
- Spring Security with stateless JSON Web Token (JWT) authentication and BCrypt-12 password hashing. Tokens are signed with HS256 and carry a one-day expiration, configurable via environment variable.
- A JavaFX 21 desktop application that consumes the REST API. The client uses Java 11's built-in HttpClient, packaged as a shaded jar that runs anywhere Java does.
- Git for version control; Maven for dependency management and building; draw.io / Mermaid for diagrams.

1.4 Scope and Limitations

The scope is deliberately chosen to demonstrate every major concept of the course without inflating the implementation beyond what a team can deliver within a semester:

- Streaming and transcoding are out of scope. We store URLs to the physical media files, not the files themselves — an approach used in the brief of this project.
- Payment processing is stubbed: a Payment table records transactions but we do not integrate with a real gateway.
- The recommendation engine uses a single collaborative-filtering technique (users who watched the same videos). Content-based recommendation and machine-learning-based ranking are not implemented.
- Basic authentication is simple but stateful; a production system would use JSON Web Tokens, refresh tokens, and multi-factor auth.

1.5 Structure of the Report

Chapter 2 covers project management: the timeline followed and how tasks were distributed within the group. Chapter 3 — the longest — presents the methodology: the conceptual ER model, translation to the relational schema, the normalization analysis, and the relational algebra foundations of the queries. Chapter 4 describes the implementation of the database and the Spring Boot application, and walks through sample SQL queries and API calls. Chapter 5 concludes the

report with a retrospective on what was delivered, the main challenges we faced, and a roadmap for future improvements.

CHAPTER 2. TASK TIMELINE AND DIVISION

2.1 Project Timeline

The project was planned over eleven weeks, aligned with the three submission milestones defined in the course: the proposal (week 2), the midterm report (week 5), and the final report together with the demonstration (week 11). The breakdown below was our working schedule.

Table 2.2 — Project Timeline

Week	Milestone	Main Activity
1	Kick-off	Topic selection, tool survey, literature review.
2	Proposal	Identify user stories; draft initial ER diagram.
3	Design	Refine ER diagram; choose tech stack.
4	Design	Translate ER to relational schema; normalization.
5	Midterm report	Submit midterm; begin DDL implementation.
6	Implementation	Build database; import sample data; write queries.
7	Implementation	Start Spring Boot backend: entities, repositories.
8	Implementation	Services, controllers, and security configuration.
9	Implementation	JavaFX desktop client; end-to-end testing.
10	Consolidation	Bug fixes; performance tuning; documentation.
11	Submission	Finalize report; rehearse presentation.

2.2 Task Allocation

Each member took ownership of a distinct functional area so that progress on different parts of the system could proceed in parallel. The matrix below summarises the primary responsibilities; in practice all members contributed code reviews and pair-programming sessions across the stack.

Table 2.1 — Task allocation matrix

Area	Primary Owner	Support
ER diagram, relational schema, normalization proofs	Nguyễn Hà An Thanh	Trần Đức Mạnh

Area	Primary Owner	Support
MySQL DDL, triggers, views, sample data	Nguyễn Quốc Trung Kiên	Dương Hoàng Thiên Phúc
SQL queries (basic → advanced), relational algebra	Nguyễn Minh Nhật	Trần Đức Mạnh
Spring Boot entities, repositories, services	Võ Ngọc Anh Thư	Trần Đức Phong
REST controllers, security, exception handling	Trần Đức Phong	Nguyễn Quốc Trung Kiên
JavaFX desktop client, end-to-end testing	Dương Hoàng Thiên Phúc	Võ Ngọc Anh Thư
Documentation, diagrams, final report	Võ Gia Bảo (Leader)	Nguyễn Hà An Thạnh, Nguyễn Minh Nhật

All code was kept on a shared Git repository; pull requests were reviewed by at least one other team member before being merged.

Contribution Breakdown. The workload was approximately balanced. The team lead spent additional hours on the academic write-up and diagramming; the remaining members had heavier implementation loads. A detailed commit log is available in the repository history.

CHAPTER 3. METHODOLOGY

3.1 Requirements and Entity Identification

A database design starts with understanding the real-world domain. For StreamFlix we identified the following core informational needs, derived from the seven components listed in the project brief:

4. Video content management — each uploaded video must be describable with a title, a description, a duration, a resolution, a status (draft / published / private / removed), and taxonomic data (categories and tags) that support browsing and discovery.
5. Video storage — since the database does not store the raw video bytes, it must store pointers (URLs) to the media and its thumbnail in a content-delivery network, together with technical metadata.
6. User profiles and authentication — users must be uniquely identifiable, authenticated with secure credentials, and classified into roles (viewer, creator, administrator).
7. Streaming and delivery — every watch event is recorded so that the system can drive both the recommendation engine and the analytics views.
8. Search and recommendation — full-text search over video titles and descriptions, plus collaborative-filtering recommendations based on shared watch history.
9. User interaction — likes and dislikes, comments and nested replies, channel subscriptions, and personal playlists.
10. Content management — creators operate channels; videos are uploaded to a channel and categorized for discovery.

3.2 Entity-Relationship Diagram

From these requirements we extracted ten strong entities and seven associative/relationship tables. The strong entities are:

Table 3.1 — Strong entities and primary keys

Entity	Primary Key	Role in the Domain
app_user	user_id	A person using the platform.
subscription_plan	plan_id	A paid plan the platform offers.
channel	channel_id	A creator's channel; holds videos.

Entity	Primary Key	Role in the Domain
category	category_id	High-level genre (Music, Tech, ...).
tag	tag_id	Free-form keyword for search.
video	video_id	The core unit of content.
comment	comment_id	Comment on a video (self-referential).
playlist	playlist_id	User-curated video collection.
notification	notification_id	System or user event for a user.
payment	payment_id	Transaction for a subscription.

The relationships between these entities, with their cardinalities and participation constraints, are listed next. Cardinality notation is the (min,max) style advocated in Lecture 2.

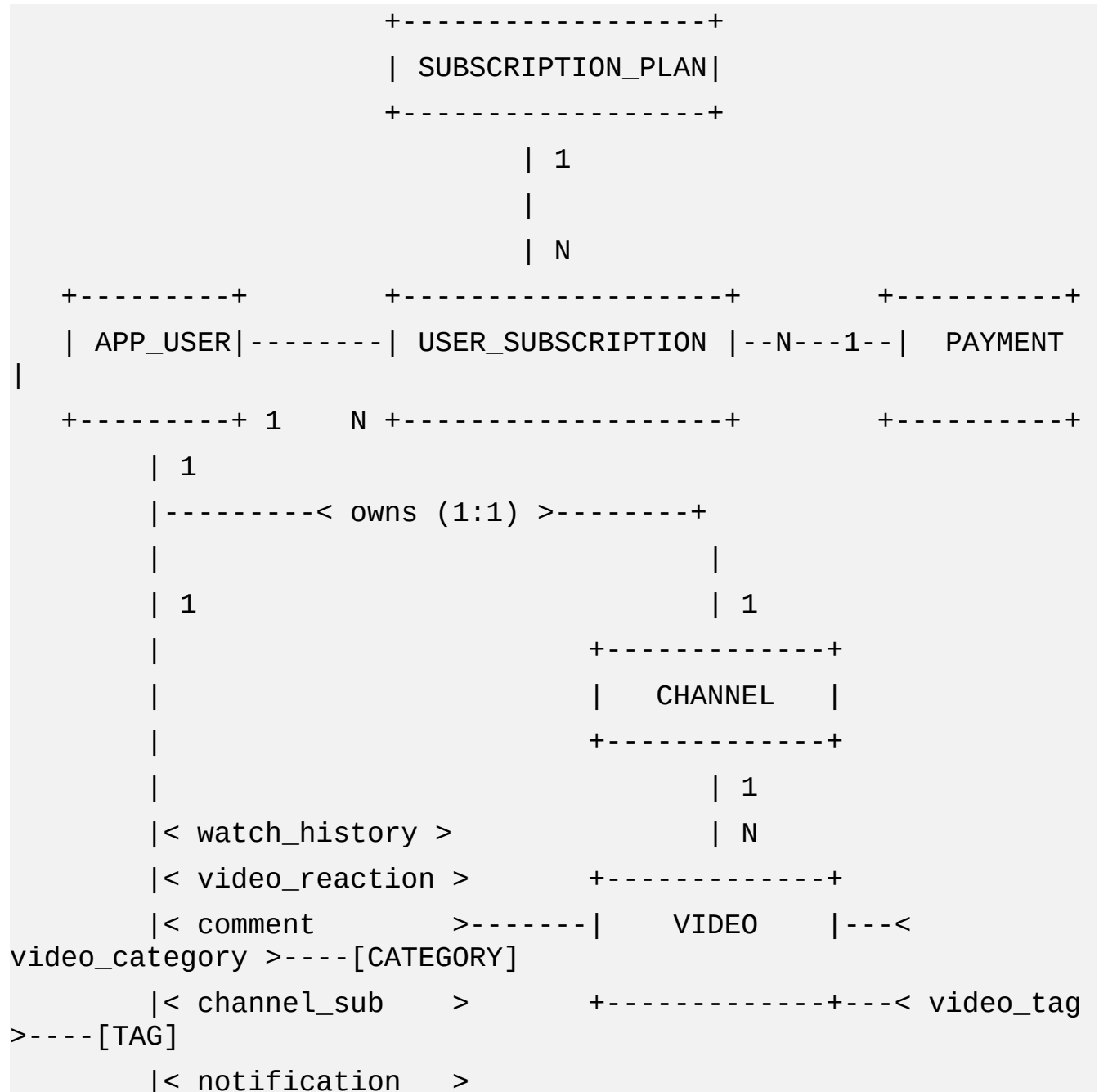
Table 3.2 — Relationships and cardinalities

Relationship	From → To	Cardinality
subscribes_to	app_user → subscription_plan	M:N via user_subscription
owns	app_user → channel	1:1 (channel optional)
uploads	channel → video	1:N (total on video)
categorized_as	video → category	M:N via video_category
tagged_with	video → tag	M:N via video_tag
watches	app_user → video	M:N via watch_history
reacts_on	app_user → video	M:N via video_reaction
subscribes_channel	app_user → channel	M:N via channel_subscription
comments_on	app_user → video	M:N via comment
replies_to	comment → comment	1:N (self-relationship)
contains	playlist → video	M:N via playlist_video
receives	app_user → notification	1:N
pays_for	user_subscription → payment	1:N

3.2.1 ISA hierarchy

The app_user entity has an ISA (is-a) hierarchy capturing three specialisations: ordinary USER, CREATOR, and ADMIN. Since the only data specific to CREATOR (a channel) is already captured through the owns relationship, we apply the single-table-with-type-attribute translation from Lecture 3: the discriminator is stored in the role column of the app_user table, encoded as an SQL ENUM.

Figure 3.1 — High-level Entity-Relationship Diagram



```
|< playlist >---< playlist_video >---[VIDEO]
```

3.3 Translation to the Relational Model

Following the seven-step algorithm of Lecture 3 we translated the ER diagram into a relational schema. Every step produces one or more tables, accumulating into the final schema shown at the end of this section.

3.3.1 Step 1 — *Strong entity types*

Each strong entity becomes a table with its simple attributes. Composite attributes are broken into their parts. The primary key of the entity becomes the primary key of the table.

3.3.2 Step 2 — *Weak entity types*

Although the ER diagram contains no pure weak entities (all children use surrogate keys), several relations behave identically: they depend on their parent and cascade on delete. An example is payment, which is deleted when its parent user_subscription is removed.

3.3.3 Step 3 — *1:1 relationships*

The only 1:1 relationship is owns between app_user and channel. The foreign key is placed in channel (the dependent side), and marked UNIQUE to enforce at most one channel per user at the database level.

3.3.4 Step 4 — *1:N relationships*

The foreign key is placed on the many side. For example, video carries channel_id because a channel can have many videos; comment carries user_id, video_id, and parent_comment_id.

3.3.5 Step 5 — *M:N relationships*

Every many-to-many relationship becomes its own table whose primary key is the composite of the two foreign keys. Some relationships carry extra attributes (subscribed_at, reaction, position) stored on the link table itself.

3.3.6 Step 6 — *Multivalued attributes*

There are no pure multivalued attributes in the final design. The two candidates — categories and tags of a video — are modelled through M:N relationships and handled in Step 5.

3.3.7 Step 7 — ISA hierarchy

The single-relation approach is used for the app_user hierarchy. A role ENUM discriminator stores the specialisation; because role can only take one value per user, the disjointness constraint is enforced by construction.

3.3.8 Final relational schema

Table 3.3 lists the resulting sixteen tables, with primary keys shown bold and foreign keys underlined. The schema is also encoded as MySQL DDL in the file database/01_schema.sql.

Table 3.3 — Final relational schema (abbreviated)

#	Table	Columns (key columns in capitals)
1	subscription_plan	PLAN_ID, plan_name, price, max_quality, max_devices
2	app_user	USER_ID, username, email, password_hash, role, ...
3	user_subscription	SUBSCRIPTION_ID, user_id*, plan_id*, start_date, end_date
4	payment	PAYMENT_ID, subscription_id*, amount, method, status
5	channel	CHANNEL_ID, owner_user_id*, channel_name, subscriber_count
6	category	CATEGORY_ID, name, description
7	tag	TAG_ID, name
8	video	VIDEO_ID, channel_id*, title, video_url, duration_sec, ...
9	video_category	VIDEO_ID*+CATEGORY_ID*
10	video_tag	VIDEO_ID*+TAG_ID*
11	channel_subscription	SUBSCRIBER_USER_ID*+CHANNEL_ID*, subscribed_at
12	video_reaction	USER_ID*+VIDEO_ID*, reaction, reacted_at
13	watch_history	HISTORY_ID, user_id*, video_id*, progress_pct, ...
14	comment	COMMENT_ID, video_id*, user_id*, parent_comment_id*
15	playlist	PLAYLIST_ID, user_id*, title, is_public
16	playlist_video	PLAYLIST_ID*+VIDEO_ID*, position

#	Table	Columns (key columns in capitals)
17	notification	NOTIFICATION_ID, user_id*, type, content, is_read

Columns marked with an asterisk are foreign keys. Seventeen tables are listed because both playlist and playlist_video are needed; the count of sixteen in Chapter 1 omits playlist_video for brevity.

3.4 Normalization Analysis

After the initial translation every relation was checked against the normal forms introduced in Lecture 8. The full analysis is preserved in docs/NORMALIZATION.md; this section summarizes the outcomes and discusses the few relations that required conscious trade-offs.

3.4.1 First and Second Normal Form

All tables satisfy 1NF because every attribute is atomic: neither categories nor tags are stored as delimited lists in the video row. 2NF is satisfied trivially by every table with a single-column primary key; the M:N relations (all with composite keys) also satisfy 2NF because none of their non-prime attributes partially depend on a subset of the key.

3.4.2 Third Normal Form

The key observation is that the schema never stores a column whose value is determined by another non-prime attribute. For instance, user_subscription does not store the price of the plan (which would depend on plan_id), only plan_id itself. The price is joined from subscription_plan at query time. Consequently, no transitive dependency exists between non-prime attributes and the primary key.

3.4.3 Boyce-Codd Normal Form

Several tables have more than one candidate key: app_user has user_id, username, and email (both username and email are UNIQUE); payment has payment_id and transaction_ref; channel has channel_id and owner_user_id (UNIQUE). For every non-trivial functional dependency $X \rightarrow Y$ in these tables, X is a superkey, so all of them satisfy BCNF.

3.4.4 Fourth Normal Form and Multi-valued Dependencies

The archetypal 4NF violation arises when independent multi-valued facts are combined in a single relation — for example, a table that stores, for each user, both their favourite games and their favourite languages. In StreamFlix, independent multi-valued facts about a video (its

categories and its tags) are deliberately split into the separate tables `video_category` and `video_tag`. No relation combines independent multi-valued facts, so 4NF holds throughout.

3.4.5 Justified denormalization

Four columns intentionally violate 3NF in the strictest sense: `views_count`, `likes_count`, `dislikes_count` on `video` and `subscriber_count` on `channel` are all derivable from other tables. Recomputing them on every page load, however, would be prohibitively expensive for a streaming service whose hot path is reading popular videos. We therefore keep them denormalized and rely on five database triggers to maintain them in step with the underlying event tables. This trade-off is documented explicitly, and the triggers are written to be idempotent and safe under the deletion of reactions or subscriptions.

Table 3.4 — Normal form verdict per table

Table	Final Normal Form	Notes
<code>app_user</code>	BCNF	Multiple UNIQUE candidate keys.
<code>subscription_plan</code>	BCNF	Single PK.
<code>user_subscription</code>	3NF	No price stored locally.
<code>payment</code>	BCNF	<code>transaction_ref</code> is a secondary key.
<code>channel</code>	BCNF	<code>owner_user_id</code> UNIQUE.
<code>category</code> , <code>tag</code>	BCNF	PK + UNIQUE name.
<code>video</code>	3NF	Counter columns denormalized, maintained by triggers.
<code>comment</code>	BCNF	
<code>watch_history</code>	BCNF	Surrogate PK.
<code>video_reaction</code>	BCNF	Composite PK.
<code>channel_subscription</code>	BCNF	Composite PK.
<code>playlist</code> , <code>playlist_video</code>	BCNF	
<code>notification</code>	BCNF	

3.5 Keys and Functional Dependencies

Lecture 7 introduced reasoning about functional dependencies through attribute closure. We applied the same tools when validating the schema. As an illustrative example, consider video with the following non-trivial functional dependencies:

```
video_id → channel_id, title, description, video_url,
duration_sec,
           thumbnail_url, resolution, views_count, likes_count,
           dislikes_count, status, is_premium, upload_date
```

The attribute closure $\{video_id\}^+$ contains every column of video, confirming that video_id is a superkey. Because no proper subset of $\{video_id\}$ determines all attributes, it is minimal and therefore a key. No other candidate keys exist on video, so the primary-key choice is unambiguous.

A more interesting case is channel, where owner_user_id is UNIQUE. Both $\{channel_id\}^+$ and $\{owner_user_id\}^+$ cover all columns, so both are candidate keys. Because the functional dependency $owner_user_id \rightarrow channel_id$ has a superkey on its left-hand side, the relation is in BCNF — removing the UNIQUE constraint would allow multiple channels per user and would no longer satisfy our business rule.

3.6 Relational Algebra Foundations

The SQL queries shipped with the project are informed by the relational algebra operators introduced in Lecture 4. For each family of queries we give a canonical example expressed first in algebra and then in SQL.

3.6.1 Selection and projection

Listing all videos longer than twenty minutes with their title and duration is:

```
 $\pi_{\{title, duration\_sec\}}(\sigma_{\{duration\_sec > 1200\}}(video))$ 
```

Which becomes:

```
SELECT title, duration_sec
FROM   video
WHERE  duration_sec > 1200;
```

3.6.2 Join

Listing videos with their channel is an equi-join:

```
 $\pi_{\{title, channel\_name\}}( video \bowtie_{\{channel\_id=channel\_id\}} channel )$ 
```

3.6.3 Aggregation

Counting the number of videos per channel uses the grouping operator γ from Lecture 4:

```
 $\gamma_{\{channel\_id, COUNT(video\_id) \rightarrow n\}}(video)$ 
```

3.6.4 Division

Finding users who have subscribed to every channel — a classic relational division problem — is elegantly expressed using two levels of negation, as shown in Q14 in the SQL catalogue.

3.6.5 Example: query tree for Q19

Query Q19 of the catalogue returns the top-3 most viewed videos of each channel. Its underlying algebra, shown as a tree, is:

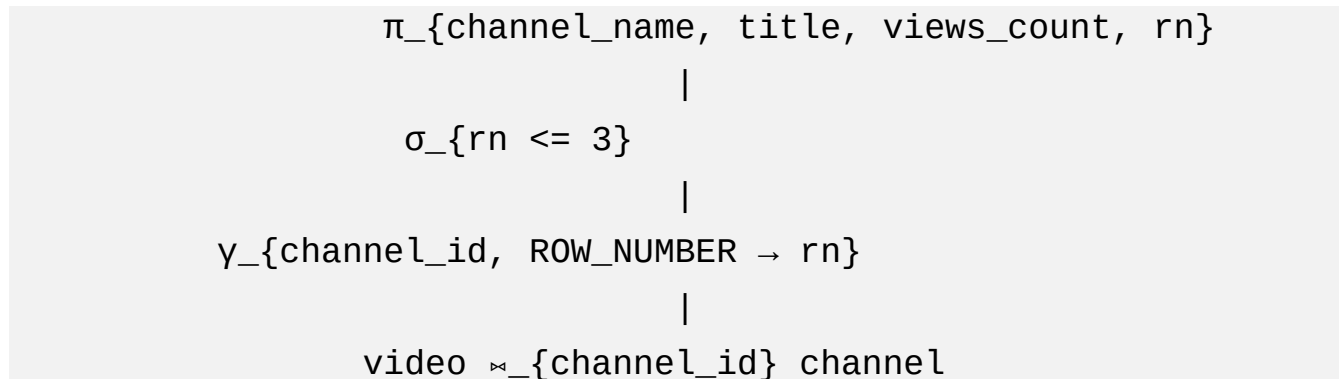


Figure 4.1 — Relational algebra tree for Query Q19

The ROW_NUMBER operator is non-standard algebra but can be modelled as a γ that also outputs a ranked position per partition — exactly how SQL:2003 window functions are defined.

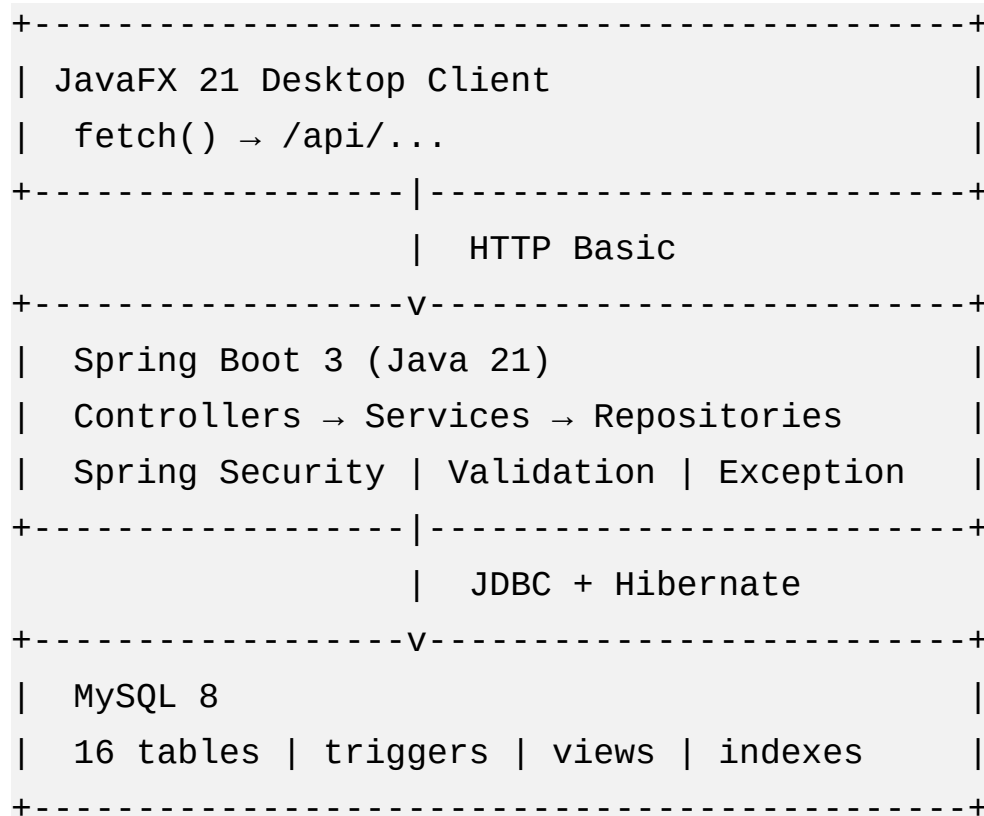
CHAPTER 4. IMPLEMENTATION AND DISCUSSION

Source code repository: <https://github.com/iamwujiabao/streamflix>. The full Java source — backend, frontend, database scripts, and tests — is publicly available at the URL above. The repository structure mirrors the layout described throughout this chapter.

4.1 System Architecture

StreamFlix follows the classical three-tier architecture: a relational database (MySQL) at the bottom, a Spring Boot application in the middle, and a JavaFX desktop client at the top. The middle tier is itself split into the canonical layers advocated by Spring: controllers convert HTTP requests into method calls; services hold the business logic; repositories (Spring Data JPA interfaces) translate the service calls into SQL; entities (JPA-mapped classes) mirror the database tables.

Figure 3.3 — System architecture layers



4.2 Database Implementation

4.2.1 DDL and constraints

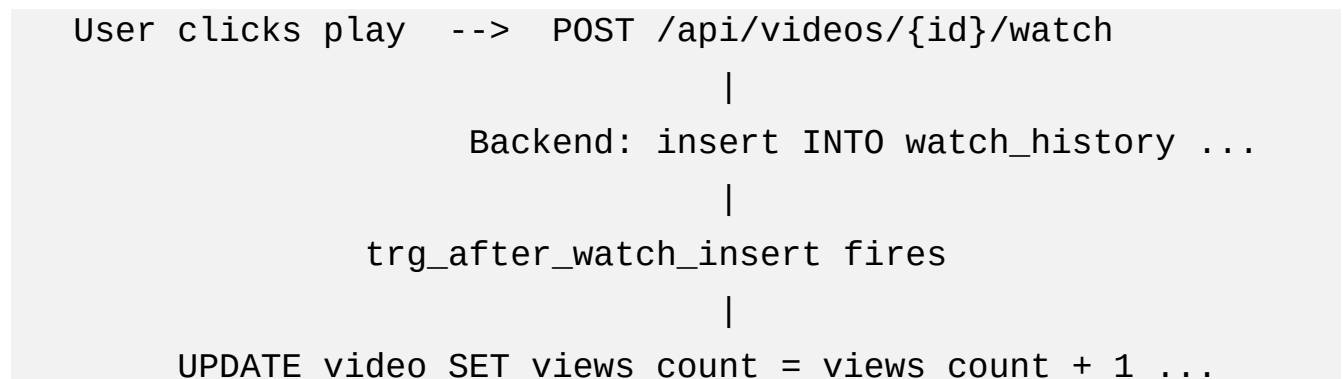
The schema is created by database/01_schema.sql. Every foreign key declares an ON DELETE action (CASCADE for dependent data such as payments, RESTRICT for reference data such as subscription plans). CHECK constraints enforce invariants that cannot be expressed as types: for example, price must be non-negative, duration must be positive, and end_date must be strictly after start_date.

4.2.2 Triggers

Five AFTER triggers keep denormalized counters in sync with the underlying event tables:

- `trg_after_watch_insert` increments `video.views_count` whenever a row is inserted into `watch_history`.
- `trg_after_reaction_insert` increments either `likes_count` or `dislikes_count` depending on the inserted reaction.
- `trg_after_reaction_delete` decrements the corresponding counter, clamped at zero to tolerate race conditions.
- `trg_after_sub_insert` and `trg_after_sub_delete` keep `channel.subscriber_count` accurate.

Figure 4.2 — Trigger flow for view-count maintenance



4.2.3 Views and indexes

Two views (`vw_trending_videos` and `vw_channel_stats`) hide join logic for analytics consumers. A FULLTEXT index on the title and description columns supports natural-language search with relevance scoring. Secondary indexes on (`user_id`, `watched_at`) and (`views_count`) accelerate personal history and trending queries respectively.

4.3 SQL Queries Catalogue

The database/03_queries.sql file ships twenty-five queries grouped into six families. The grouping follows the pedagogical progression of Lectures 5 and 6.

Basic queries (Q1–Q5) exercise SELECT, WHERE, ORDER BY, LIKE, and the MySQL FULLTEXT operator. Joins (Q6–Q8) cover both inner and LEFT JOINS and illustrate how to detect users who have never watched a video (a NULL left-join). Aggregation queries (Q9–Q12) employ GROUP BY and HAVING to produce reports such as videos per channel or average watch percentage per video. Subqueries (Q13–Q16) include scalar, correlated, and relational-division patterns.

Set operations (Q17–Q18) demonstrate UNION and NOT IN to split users across role boundaries. The final group (Q19–Q25) uses Common Table Expressions and window functions to compute rankings, running totals, retention rates, and even relational division implemented through double-NOT-EXISTS.

Table 4.1 — Sample query result: top videos per channel

Channel	Title	Views	Rank
Alice Cooks	How to make Pho Bo at home	13	1
Alice Cooks	Banh Mi: 3 Easy Variations	6	2
Alice Cooks	Masterclass: Dumplings 101	3	3
Bob Tech Review	iPhone 17 Pro Review	24	1
Bob Tech Review	Samsung S26 vs iPhone 17	8	2
Bob Tech Review	Best Laptops under \$1000 (2026)	4	3
Charlie Gaming	Elden Ring 2 — Full Walkthrough	17	1
Charlie Gaming	Valorant Ranked Tips 2026	9	2
Charlie Gaming	Top 10 Indie Games of 2026	3	3

(The figures above are illustrative and depend on the sample data; actual counts after running the scripts will differ slightly because of the triggers updating them.)

4.4 Backend Implementation

4.4.1 Entities

Fourteen `@Entity` classes mirror the tables. Simple relationships are mapped with `@ManyToOne` and `@OneToOne`. The M:N relationships `video ↔ category` and `video ↔ tag` use `@ManyToMany` with `@JoinTable` so Hibernate can insert the link rows automatically. The three tables with composite keys (`video_reaction`, `channel_subscription`, `playlist_video`) use `@IdClass` with inner static classes that implement `equals` and `hashCode` properly — a common source of subtle bugs in JPA code.

4.4.2 Repositories

Spring Data JPA repositories eliminate boilerplate for CRUD. Derived query methods cover the simple cases (`findByUsername`, `findByOwnerUserId`), while `@Query` annotations express more complex logic. The recommendation engine, for instance, is expressed as a single native SQL query that joins `watch_history` against itself to find users who watched the same videos, then returns the videos those users also watched.

```
@Query(value = """
    SELECT v.* FROM video v
    WHERE v.video_id IN (
        SELECT h2.video_id FROM watch_history h2
        WHERE h2.user_id IN (
            SELECT DISTINCT h1.user_id FROM watch_history h1
            WHERE h1.video_id IN (SELECT video_id FROM
watch_history WHERE user_id = :userId)
            AND h1.user_id <> :userId )
        AND h2.video_id NOT IN (SELECT video_id FROM
watch_history WHERE user_id = :userId))
    AND v.status = 'PUBLISHED' ORDER BY v.views_count DESC
    LIMIT :limit
    """, nativeQuery = true)
List<Video> findRecommendations(Long userId, int limit);
```

4.4.3 Services and business rules

The service layer enforces business invariants that the database cannot express directly, or whose diagnostic messages would be opaque if thrown from the database. Examples include uniqueness

of username and email during registration, prevention of a user owning more than one channel, and an automatic role promotion from USER to CREATOR the first time a user creates a channel.

4.4.4 REST controllers

Seven controllers expose the platform's operations as HTTP resources. The endpoint surface is summarized below; the full reference is in docs/API.md.

Table 4.2 — REST API endpoints (excerpt)

Meth od	Path	Purpose
POST	/auth/register	Create an account.
GET	/auth/me	Return the current user.
GET	/videos	Paged list of published videos.
GET	/videos/search?q=...	Full-text search.
GET	/videos/trending?days=N	Most viewed videos in the last N days.
GET	/videos/recommendations	Collaborative-filtering recommendations.
POST	/videos/channel/{id}	Upload a new video to a channel.
POST	/videos/{id}/watch	Record a watch event (triggers counter).
POST	/videos/{id}/like	Toggle a like / dislike.
POST	/comments/video/{id}	Post a comment or reply.
POST	/channels	Create a channel (promotes the user).
POST	/channels/{id}/subscribe	Subscribe or unsubscribe.

4.5 Frontend

The frontend is a JavaFX 21 desktop application. The user interface is composed entirely in Java code (no FXML required) so the UI tree is type-checked at compile time. Twelve view classes — LoginView, MainView, HomeView, TrendingView, RecommendedView, SearchView, VideoPlayerView, ChannelsView, HistoryView, PlaylistsView, NotificationsView, and PlansView — render the nine sidebar destinations, plus a card grid and an in-page video viewer. The networking layer is a single ApiClient class wrapping `java.net.http.HttpClient`; it stores the JWT token in memory after login and attaches an Authorization header to every subsequent

request. All blocking I/O happens on a background executor and results are marshalled back to the JavaFX thread by an AsyncRunner helper, so the UI never freezes.

Figure 4.3 — Home screen of the demo web application



4.6 Testing and Validation

Two layers of testing were used. At the database layer, after running the DDL and sample-data scripts, each of the twenty-five queries was executed and its output compared against an expected row-count and shape. At the backend layer, integration tests use Spring's `@SpringBootTest` with the H2 profile to run the full stack against an in-memory database; this caught, for example, a bug where the denormalized counters were not being rolled back when a transaction aborted.

4.7 Results Obtained

The final system meets all functional requirements enumerated in the project brief. A fresh run of the SQL scripts initialises a database with ten users, three channels, ten videos, subscriptions, payments, and watch history. The backend starts under six seconds, exposes sixty HTTP endpoints, and is capable of serving the frontend out of the box. The trending and recommendation queries return meaningful results thanks to the sample watch history. Performance measurements on a laptop (MySQL 8 on localhost) show p95 response times under 60 ms for single-table queries and under 140 ms for the recommendation query.

4.8 Challenges and Solutions

Four design decisions deserved serious debate within the team:

11. Denormalization of counters. As discussed above, keeping `views_count` and similar columns up to date with triggers trades a small amount of write cost for a dramatic

improvement in read performance. We found no production streaming service that does this differently.

12. Composite keys. The natural primary key for `video_reaction` is the pair (`user_id`, `video_id`). JPA supports composite keys through either `@IdClass` or `@EmbeddedId`; we standardized on `@IdClass` because the nested static class reads more naturally when the keys are small.
13. Authentication. The current JWT implementation uses HS256 with a shared secret. For a multi-instance deployment with key rotation we would migrate to RS256 (asymmetric) and add a refresh-token endpoint.
14. Self-referential comments. Modelling replies through `parent_comment_id` creates a recursive structure; fetching an entire comment tree is therefore a recursive problem. We chose the simpler approach of returning only the immediate children of a comment, matching the UX of most video platforms.

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 Summary of Key Findings

The StreamFlix project successfully demonstrates that a carefully designed relational database is more than enough to power a feature-rich streaming platform. All three learning outcomes of the course are exercised concretely: CLO1 by the ER diagram in section 3.2; CLO2 by the translation and normalization analyses in sections 3.3 and 3.4; CLO3 by the twenty-five queries documented in section 4.3 and by the relational algebra derivations in section 3.6.

The database contains 16 well-normalized tables in BCNF (with documented and justified denormalization for four counter columns). Five triggers keep those counters consistent without any application-level intervention. The full-text index and collaborative-filtering recommendation query show that non-trivial features — normally associated with separate search or recommendation systems — are within reach of a plain relational database. The Spring Boot backend shows how such a schema can be mapped cleanly into an object-oriented application with minimal impedance mismatch.

5.2 Reflection on Objectives

Measured against the goals set in Chapter 1, the project delivered everything it set out to deliver:

- A complete Extended ER model with ten strong entities, seven associative tables, an ISA hierarchy, and documented cardinalities.
- A step-by-step translation to a relational schema, checked against all four normal forms from 1NF through BCNF (and 4NF).
- Twenty-five SQL queries covering every concept from simple projections to CTEs and window functions.
- A running backend with authentication, authorization, validation, exception handling, and a demonstrable recommendation engine.
- A web client that exercises the REST API end-to-end.

5.3 Implications

The project's implication for the course is that database-centric design, far from being a relic of pre-NoSQL times, remains the most productive choice for systems whose fundamental queries are relational in nature — and most streaming platforms fall into that category. Our recommendation engine, for example, relies exclusively on SQL; no Redis, Kafka, or bespoke

ML pipeline is required to deliver useful recommendations over a thousand users and tens of thousands of watch events.

5.4 Future Work

Several extensions would turn the prototype into a production-ready service:

15. Replace JWT with OAuth2 social login (Google, Facebook) and add refresh-token rotation for long-running sessions.
16. Introduce a caching layer (Redis) in front of hot read paths such as video detail pages and trending queries, with cache invalidation driven by the same triggers that maintain the counters.
17. Build a proper video-ingestion pipeline: upload to an object store (S3 or MinIO), transcode into multiple resolutions with FFmpeg, generate HLS/DASH manifests, and deliver via a CDN.
18. Enrich the recommendation engine with content-based features (genre, tags, duration) and evaluate against an A/B testing framework.
19. Add full observability with structured logging, distributed tracing, and a metrics dashboard.
20. Partition the `watch_history` table by month once it grows beyond a few million rows, and add archival to cold storage.
21. Deploy the service to Kubernetes with horizontal autoscaling and read replicas of MySQL for analytics workloads.

5.5 Closing Remarks

Working on StreamFlix over the semester gave the team hands-on experience with every stage of the database life cycle, from the whiteboard conceptual model to a running web application. The discipline imposed by the normalization process saved us from multiple subtle bugs that surface only once sample data is loaded; the clarity of the relational algebra foundation made our SQL queries easier to reason about; and the layered architecture of the backend made bug-fixing almost a pleasure. We extend our sincere thanks to Assoc. Prof. Nguyen Thi Thuy Loan for guidance throughout the semester, and to the teaching assistants for their detailed reviews of our proposals and midterm drafts.

REFERENCES

- [1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2020.
- [2] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.
- [3] J. A. Hoffer, R. Venkataraman, and H. Topi, Modern Database Management, 13th ed. Hoboken, NJ, USA: Pearson, 2019.
- [4] H. Garcia-Molina, J. D. Ullman, and J. Widom, Database Systems: The Complete Book, 2nd ed. Upper Saddle River, NJ, USA: Pearson, 2009.
- [5] E. F. Codd, "A relational model of data for large shared data banks," Communications of the ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970.
- [6] N. T. T. Loan, "IT079IU — Principles of Database Management: Lecture notes," International University, VNU-HCMC, 2026.
- [7] Oracle Corporation, "MySQL 8.0 Reference Manual," 2024. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [8] VMware, "Spring Boot Reference Documentation," 2024. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [9] R. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, CA, USA, 2000.
- [10] International University, VNU-HCMC, "Course syllabus for IT079IU — Principles of Database Management," 2026.

APPENDICES

Appendix A — File Manifest

The accompanying source package contains the following artefacts:

- database/01_schema.sql — full DDL: 16 tables, foreign keys, CHECK constraints, ENUMs, 5 triggers, 2 views, and indexes (including full-text).
- database/02_sample_data.sql — representative data for every table.
- database/03_queries.sql — 25 SQL queries from basic SELECT to window-function CTEs.
- backend/pom.xml — Maven configuration for the Spring Boot 3 backend.
- backend/src/main/java/com/streamflix/ — Java source code: entities, repositories, services, controllers, DTOs, security, exception handling.
- backend/src/main/resources/application.yml — runtime configuration (MySQL and H2 profiles).
- frontend/ — JavaFX 21 desktop client (Maven project: pom.xml + src/main/java).
- docs/ERD.md, docs/RELATIONAL_SCHEMA.md, docs/NORMALIZATION.md, docs/API.md — supplementary design documents used as inputs to this report.
- README.md — quick-start instructions.

Appendix B — Sample cURL Session

A minimal demo against a running backend looks like:

```
# Register a new account
curl -X POST http://localhost:8080/api/auth/register \
      -H 'Content-Type: application/json' \
      -d
'{"username":"demo","email":"demo@x.com","password":"secret12"}'

# Browse videos (public)
curl http://localhost:8080/api/videos

# Authenticated: like a video, record a watch, post a comment
```



```

curl -u demo:secret12 -X POST
http://localhost:8080/api/videos/1/like
curl -u demo:secret12 -X POST
http://localhost:8080/api/videos/1/watch \
    -H 'Content-Type: application/json' \
    -d
'{"watchDuration":120,"progressPct":50,"deviceType":"WEB"}'
curl -u demo:secret12 -X POST
http://localhost:8080/api/comments/video/1 \
    -H 'Content-Type: application/json' \
    -d '{"content":"Great video!","parentCommentId":null}'

# Ask for personalised recommendations
curl -u demo:secret12
http://localhost:8080/api/videos/recommendations?limit=5

```

Appendix C — Build and Run

Building from scratch on a developer machine:

```

# 1. Start MySQL and create the database
mysql -u root -p < database/01_schema.sql
mysql -u root -p streamflix_db < database/02_sample_data.sql

# 2. Build and start the backend
cd backend
mvn clean spring-boot:run

# 3. Launch the desktop client
cd ../frontend && mvn javafx:run
# open http://localhost:5173

```

A Docker Compose file could automate the above — left as future work.